S2D-03: SPL to DQL Translation

> **Series:** S2D — Splunk to Dynatrace Migration | **Notebook:** 3 of 9 | **Created:** January 2026 | **Last Updated:** 11/02/2026

Overview

This notebook provides a comprehensive guide for translating Splunk SPL (Search Processing Language) queries to Dynatrace DQL (Dynatrace Query Language). While both languages share similar concepts, their syntax and capabilities differ significantly.

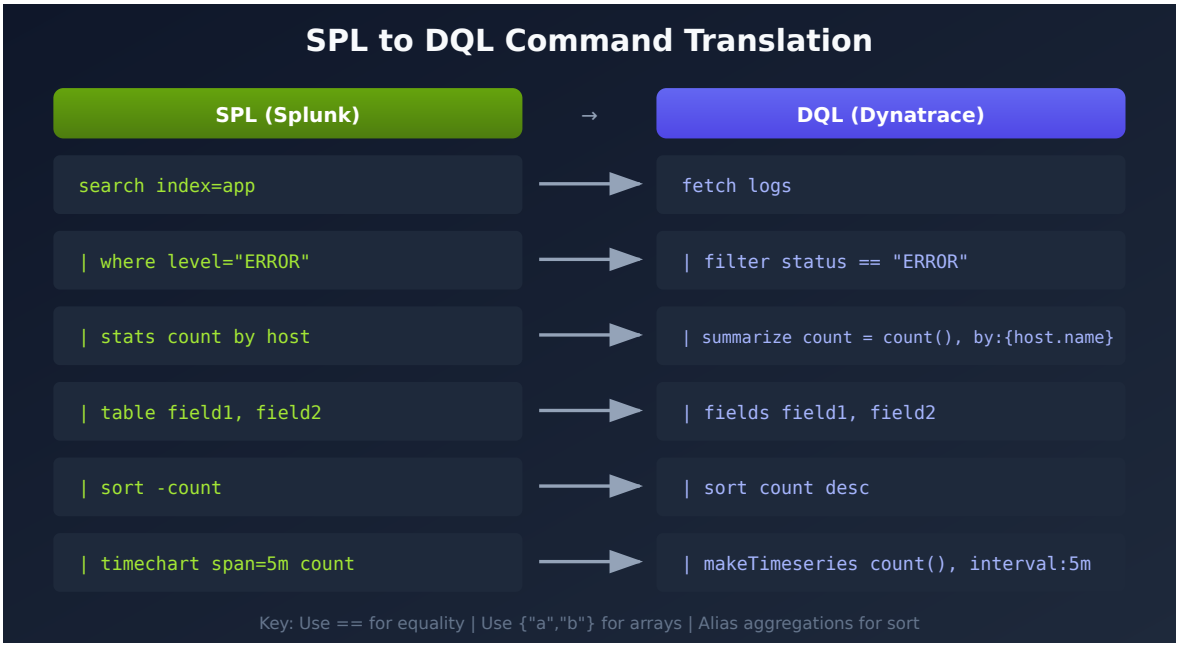


Table of Contents

1. Command Translation Reference
2. Basic Query Translation
3. Filtering Patterns
4. Aggregation Patterns

5. Field Extraction
6. Time-Series Queries
7. Key Syntax Differences

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail
Permissions	`storage:logs:read`, `storage:buckets:read`
Knowledge	Familiarity with source SPL queries

Learning Objectives

By the end of this notebook, you will be able to:

1. Map SPL commands to their DQL equivalents
2. Translate common SPL patterns to DQL
3. Handle differences in string matching and filtering
4. Convert aggregation and time-series queries
5. Apply DQL best practices during translation

Command Translation Reference

Core Commands

SPL Command	DQL Command	Description
`search index=...`	`fetch logs`	Select data source
`where field="value"`	`filter field == "value"`	Filter records (case-sensitive in both)

SPL Command	DQL Command	Description
`stats count by field`	`summarize count = count(), by: {field}`	Aggregate data
`table field1, field2`	`fields field1, field2`	Select columns
`sort -count`	`sort count desc`	Sort results
`head 10`	`limit 10`	Limit results
`eval new=field1+field2`	`fieldsAdd new = field1 + field2`	Calculate fields
`rename old AS new`	`fieldsRename old, alias:new`	Rename fields
`dedup field`	`dedup field`	Keeps the first record per value; add `sort: {…}` to choose which. Nulls differ: DQL keeps one record for null values, while Splunk drops them by default (dedup (Splunk docs): *"Default: false. All events where any of the selected fields are null are dropped."*)

String Matching

SPL Pattern	DQL Pattern	Description
`field="*error*"`	`contains(field, "error", caseSensitive:false)`	Substring
`"error"` (bare term)	`matchesPhrase(content, "error")`	Whole word or phrase
`field=error`	`matchesValue(field, "error")`	Whole value, case-insensitive like Splunk search; use `field == "error"` only for a case-sensitive match (Splunk `CASE(error)` or `where`)

SPL Pattern	DQL Pattern	Description
`NOT field="v"`	`filterOut matchesValue(field, "v")`	Keeps records that have no `field`
`field!="v"`	`filter not(matchesValue(field, "v"))`	Drops records that have no `field`
`field IN ("a","b")`	`in(field, {"a", "b"})`	Multiple values
`rex field=f "(?<name>...)"`	`parse f, "DATA:name"`	Extract patterns

Splunk search matches field values case-insensitively. The Search Manual says *"By default, searches are case-insensitive. For example, if you search for* `Error` *, any case of that term is returned, such as* `Error` *,* `error` *, and* `ERROR` *."* DQL `==` is case-sensitive, while `matchesValue` is not by default: its `caseSensitive` parameter is *"Whether the match should be done case-sensitive (default: false)."* The `where` command is different: Splunk says *"The is case-sensitive."*, so `where field="value"` translates to `==`.

Basic Query Translation

Example 1: Simple Log Search

SPL:

```
index=application sourcetype=app_logs host=app-server-01 | head 100
```

DQL:

```
// Simple log search with host filter
fetch logs, from:-1h
| filter matchesPhrase(host.name, "app-server-01")
| limit 100
```

Example 2: Error Log Count by Host

SPL:

```
index=application level=ERROR | stats count by host | sort -count
```

Splunk `level=ERROR` usually means "anything at error severity or worse". In Dynatrace that is `status == "ERROR"`, which groups SEVERE, ERROR, CRITICAL, ALERT and EMERGENCY. Use `loglevel == "ERROR"` only when you need the literal level; on a validation tenant (10/05/2026) it matched about half of the records that `status == "ERROR"` matched, because Java `SEVERE` logs fall outside it.

DQL:

```
// Error log count by host
fetch logs, from:-1h
| filter status == "ERROR"
| summarize count = count(), by:{host.name}
| sort count desc
```

Example 3: Time-Based Aggregation

SPL:

```
index=application | timechart span=5m count by level
```

DQL:

```
// Time-series count by log level
fetch logs, from:-24h
| makeTimeseries count = count(), by:{loglevel}, interval:5m
```

Filtering Patterns

Phrase Matching (Contains)

SPL:

```
index=application "connection timeout"
```

DQL:

```
// Search for phrase in log content
fetch logs, from:-1h
| filter matchesPhrase(content, "connection timeout")
| limit 50
```

Multiple Value Filter (IN)

SPL:

```
index=application level IN ("ERROR", "WARN", "FATAL")
```

Dynatrace has no `FATAL` level. The log ingest rules page lists the level values: *"It supports the following values: alert, critical, debug, emergency, error, info, none, notice, severe, warn."* Translate `FATAL` to the error-or-worse levels instead.

DQL:

```
// Filter for multiple log levels
// Dynatrace has no FATAL level; SEVERE, CRITICAL, ALERT and EMERGENCY are
error-or-worse
fetch logs, from:-1h
| filter in(loglevel, {"ERROR", "SEVERE", "CRITICAL", "ALERT", "EMERGENCY",
"WARN"})
| summarize count = count(), by:{loglevel}
```

NOT Filter

SPL:

```
index=application NOT level="DEBUG"
```

Splunk `NOT` and `!=` are not the same filter. The Search Manual: with `!=`, *"Events that do not have a value in the field are not included in the results."* With `NOT`, *"every event is returned except the events that contain the value you specify. This includes events that do not have a value in the field."* DQL draws the same line. `filter level != "DEBUG"` drops a record with no `level` (the comparison is null), and `filterOut` keeps it. So translate `NOT field="v"` with `filterOut` and `field!="v"` with `filter`, and use `matchesValue` to keep Splunk's case-insensitive match:

SPL	DQL	Record with no `level`
`NOT level="DEBUG"`	`filterOut matchesValue(loglevel, "DEBUG")`	kept
`level!="DEBUG"`	`filter not(matchesValue(loglevel, "DEBUG"))`	dropped

`loglevel` itself is always set on Dynatrace logs (records with no level carry `NONE`), so both forms return the same rows for it. The difference shows on optional fields, such as attributes that only some sources send.

DQL:

```
// Exclude DEBUG level logs - SPL: NOT level="DEBUG"
// filterOut keeps records without the field (like Splunk NOT); matchesValue is case-insensitive (like Splunk search)
fetch logs, from:-1h
| filterOut matchesValue(loglevel, "DEBUG")
| summarize count = count(), by:{loglevel}
```

Combined Conditions (AND/OR)

SPL:

```
index=application (host="app-01" OR host="app-02") AND level="ERROR"
```

DQL:

```
// Combined AND/OR conditions
fetch logs, from:-1h
| filter (matchesPhrase(host.name, "app-01") or matchesPhrase(host.name, "app-02"))
| filter status == "ERROR"
| limit 100
```

Aggregation Patterns

Multiple Aggregations

SPL:

```
index=application | stats count, avg(response_time), max(response_time) by
host
```

Splunk extracts `key=value` fields at search time, so `response_time` exists on raw events. Grail does not. Parse the field at query time, as below, or at ingest with OpenPipeline (S2D-07).

DQL:

```
// Multiple aggregations by host
// response_time is not a built-in field: parse it from the content first
fetch logs, from:-1h
| parse content, "DATA? 'response_time=' DOUBLE:response_time"
| filter isNotNull(response_time)
| summarize {
    count = count(),
    avg_response = avg(response_time),
    max_response = max(response_time)
  }, by:{host.name}
```

Conditional Count

SPL:

```
index=application | stats count(eval(level="ERROR")) as errors, count as total
by host
```

DQL:

```
// Conditional count - errors vs total
fetch logs, from:-1h
| summarize {
    errors = countIf(status == "ERROR"),
    total = count()
  }, by:{host.name}
| fieldsAdd error_rate = (toDouble(errors) / toDouble(total)) * 100
```

Field Extraction

Regex Pattern Extraction

SPL:

```
index=application | rex field=_raw "user=(?&lt;username&gt;\w+)"
```

`parse` matches from the start of the field, and `LD` stops at a line break. Lead the pattern with `DATA?` to get `rex`'s search-anywhere behaviour. FAQ-15 is the full DPL reference.

DQL:

```
// Extract username from log content
fetch logs, from:-1h
| filter matchesPhrase(content, "user=")
| parse content, "DATA? 'user=' WORD:username"
| summarize count = count(), by:{username}
| sort count desc
```

Key-Value Extraction

SPL:

```
index=application | rex field=_raw "status=(?&lt;status&gt;\d+)" | rex
field=_raw "duration=(?&lt;duration&gt;\d+)"
```

DQL:

```
// Extract multiple fields from structured log
// One parse per field, like two independent rex calls: field order does not matter
fetch logs, from:-1h
| parse content, "DATA? 'status=' INT:status"
| parse content, "DATA? 'duration=' INT:duration"
| filter isNotNull(status) and isNotNull(duration)
| summarize avg_duration = avg(duration), by:{status}
```

Time-Series Queries

Timechart Equivalent

SPL:

```
index=application level=ERROR | timechart span=1m count by host
```

DQL:

```
// Time-series error count by host
fetch logs, from:-24h
| filter status == "ERROR"
| makeTimeseries count = count(), by:{host.name}, interval:1m
```

Key Syntax Differences

Important DQL Rules

Aspect	SPL	DQL
String quotes	Single or double	Double only
Array syntax	`("a", "b")`	`{"a", "b"}`
Equality	`=` (case-insensitive in search)	`==` (case-sensitive); `matchesValue()` for case-insensitive
Named params	Positional	Required: `decimals: 2`

Aspect	SPL	DQL
Pipe symbol	`|`	`|`
Field access	`field_name`	`field.name` (nested)

Aggregation Aliasing (Critical)

In DQL, aggregations MUST be aliased if you want to reference them later:

```
// Wrong - cannot sort by count()
| summarize count(), by:{host.name}
| sort count() desc  // ERROR!

// Correct - alias the aggregation
| summarize count = count(), by:{host.name}
| sort count desc  // Works!
```

Next Steps

With your queries translated, proceed to **S2D-04: Alert Migration - Anomaly Detectors** to learn how to convert Splunk alerts to continuous Dynatrace monitoring.

References

- DQL Reference
- DQL Functions
- Log ingest rules (DT docs)
- DQL extraction and parsing commands (DT docs)
- DQL string functions (DT docs)
- Use CASE() and TERM() to match phrases (Splunk docs)
- Difference between != and NOT (Splunk docs)
- where command (Splunk docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.